

SENAI- Serviço Nacional de Aprendizagem Industrial

Aprendizagem Industrial em Programador de Sistemas da Informação

Relatório - Banco de Dados

Aluna: Paola Crai dos Santos

Matéria: Criação e Manipulação de Banco de Dados

Professor: Lucas de Mattos

Salto Veloso – SC

2026

INTRODUÇÃO

Os bancos de dados são ferramentas fundamentais para o armazenamento, organização e gerenciamento de informações em sistemas computacionais. Eles estão presentes em diversos ambientes, como empresas, escolas, hospitais, lojas e plataformas digitais, permitindo que grandes quantidades de dados sejam armazenadas e consultadas de maneira organizada e eficiente.

Neste trabalho, será apresentado o estudo e a utilização de bancos de dados relacionais, modelo que organiza as informações por meio de tabelas compostas por linhas e colunas. Esse modelo foi escolhido por permitir uma estrutura organizada para os dados e possibilitar o relacionamento entre diferentes informações, facilitando sua consulta, atualização e gerenciamento.

Durante as atividades desenvolvidas no curso de Aprendizagem Industrial em Programador de Sistemas, foram utilizados o MySQL e o MariaDB, sistemas de gerenciamento de bancos de dados que trabalham com a linguagem SQL (Structured Query Language). Por meio dessas ferramentas, foi possível criar e manipular bancos de dados, inserir informações, realizar consultas e utilizar diferentes recursos para facilitar o gerenciamento dos dados.

Ao longo deste relatório, serão apresentados alguns dos principais conceitos e recursos estudados, incluindo a criação e utilização de tabelas, Views, Stored Procedures e Triggers. Cada um desses recursos possui uma função específica e pode contribuir para tornar o banco de dados mais organizado, funcional e eficiente.

O objetivo deste trabalho é apresentar os conhecimentos adquiridos durante o estudo de banco de dados, demonstrando de forma teórica e prática como esses recursos podem ser utilizados no desenvolvimento de sistemas. Dessa forma, o relatório busca relacionar os conteúdos aprendidos em aula com sua aplicação no desenvolvimento de soluções computacionais.

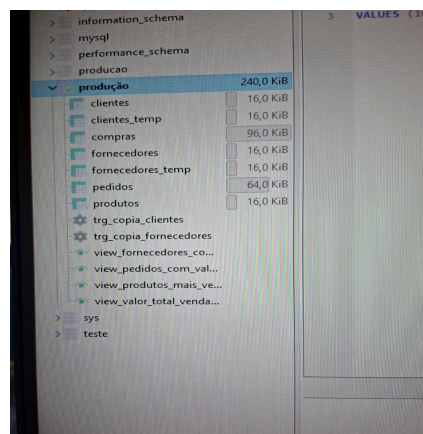


Imagem 1: estrutura do banco de dados produção no HeidiSQL

A estrutura apresentada demonstra como diferentes tabelas e recursos podem fazer parte de um mesmo banco de dados. Essa organização é fundamental para que as informações possam ser utilizadas posteriormente por consultas, Views, Stored Procedures e Triggers, que serão apresentados nas próximas partes deste relatório

TABELAS E ORGANIZAÇÕES DOS DADOS

O banco de dados utilizado no projeto segue o modelo relacional, no qual as informações são organizadas em diferentes tabelas. Cada tabela possui uma finalidade específica e é formada por colunas e registros. As colunas representam os tipos de informações que serão armazenadas, enquanto os registros correspondem aos dados inseridos no banco.

No banco de dados produção, foram utilizadas tabelas relacionadas a diferentes partes do sistema, como clientes, fornecedores, compras, pedidos e produtos. Essa divisão permite organizar as informações de acordo com sua finalidade, facilitando a localização, consulta e alteração dos dados.

A tabela clientes é responsável por armazenar informações dos clientes cadastrados. Entre os campos presentes estão o identificador (id), nome, CPF ou CNPJ, endereço, número, bairro, CEP, referência e telefone. O campo id é utilizado para identificar cada cliente de maneira única dentro da tabela.

A tabela fornecedores possui a função de armazenar os dados dos fornecedores. Assim como na tabela de clientes, são armazenadas informações de identificação e contato, permitindo manter os dados dos fornecedores organizados dentro do banco.

A tabela produtos é utilizada para armazenar as informações dos produtos disponíveis no sistema. Já a tabela pedidos registra informações relacionadas aos pedidos realizados. A tabela compras também faz parte da estrutura do banco, sendo utilizada para registrar informações relacionadas às compras.

A utilização de tabelas separadas é importante porque evita que todas as informações sejam armazenadas em um único lugar. Dessa forma, os dados ficam mais organizados e podem ser relacionados quando necessário. Por exemplo, informações de um produto podem ser utilizadas junto com informações de um pedido sem que seja necessário repetir todos os dados do produto em cada registro.

Outro conceito importante é a utilização de chaves. A chave primária é responsável por identificar de forma única cada registro de uma tabela. No banco, o campo id pode exercer essa função. Já as chaves estrangeiras podem ser utilizadas para estabelecer relações entre diferentes tabelas, permitindo que os dados sejam conectados.

Além das tabelas principais, o banco possui algumas tabelas com o sufixo `_temp`, como `clientes_temp` e `fornecedores_temp`. Essas tabelas podem ser utilizadas em processos temporários ou em conjunto com outros recursos do banco, como os Triggers. Isso demonstra que o banco foi estruturado não apenas para armazenar informações, mas também para realizar diferentes operações de manipulação e organização dos dados.

A organização por tabelas também facilita a utilização da linguagem SQL, permitindo realizar operações como inserir novos registros, consultar informações, alterar dados existentes e excluir

registros. Esses comandos são importantes para o funcionamento de sistemas que precisam trabalhar constantemente com informações armazenadas em bancos de dados.

VIEWS - QUAIS OS DADOS?

As Views, também chamadas de visões, são recursos utilizados em bancos de dados relacionais para facilitar a consulta e a visualização de informações. Uma View pode ser entendida como uma tabela virtual, criada a partir de uma consulta SQL. Diferentemente de uma tabela comum, ela não precisa armazenar novamente os dados, pois apresenta informações que já estão armazenadas em outras tabelas.

No banco de dados produção, foram criadas diferentes Views com objetivos específicos. Entre elas estão `view_fornecedores_co...`, `view_pedidos_com_val...`, `view_produtos_mais_ve...` e `view_valor_total_vend...`. Cada uma realiza uma consulta de acordo com os critérios definidos em seu código, permitindo obter informações específicas sem a necessidade de realizar manualmente uma consulta completa todas as vezes.

A utilização de Views facilita principalmente a organização das consultas. Em um banco de dados com várias tabelas e muitos registros, uma consulta pode precisar utilizar diferentes condições, cálculos e relacionamentos. Ao criar uma View, essa consulta fica armazenada e pode ser utilizada posteriormente de maneira mais simples.

Um exemplo estudado foi a View relacionada aos pedidos e produtos. Nessa consulta, são utilizadas informações presentes nas tabelas pedidos e produtos. A partir desses dados, é possível realizar cálculos e selecionar somente os registros que atendem aos critérios estabelecidos.

Nesse caso, uma das consultas trabalhadas calculava o valor de cada pedido, utilizando a quantidade de produtos e o valor correspondente. Depois, os resultados poderiam ser comparados com a média dos valores dos pedidos, permitindo identificar aqueles que apresentavam valores acima da média.

Esse tipo de consulta demonstra uma das principais utilidades das Views: apresentar somente as informações necessárias para uma determinada análise. Em vez de consultar todas as informações existentes nas tabelas, o usuário pode acessar diretamente o resultado que foi definido na View.

As Views também podem contribuir para a organização e segurança das informações. Como elas permitem selecionar somente determinadas colunas e registros, é possível apresentar apenas os dados necessários para uma determinada consulta, sem precisar disponibilizar toda a estrutura das tabelas originais.

A criação de uma View é realizada utilizando o comando `CREATE VIEW`, seguido pelo nome que será atribuído à consulta e pelo comando `SELECT` responsável por definir quais dados serão apresentados. Dessa forma, a View funciona como uma consulta previamente organizada que pode ser reutilizada sempre que necessário.

Outro ponto importante é que as informações apresentadas pela View acompanham os dados das tabelas utilizadas em sua consulta. Portanto, quando os dados das tabelas são alterados, a consulta da View pode apresentar os resultados atualizados de acordo com essas alterações.

No desenvolvimento das atividades, as Views permitiram trabalhar com diferentes informações do banco, como fornecedores, produtos, pedidos e valores de vendas. Assim, esse recurso foi importante para praticar consultas mais elaboradas e compreender como diferentes tabelas podem ser utilizadas em conjunto.

Dessa maneira, respondendo à pergunta proposta no trabalho — “Views: quais os dados?” —, as Views apresentam os dados definidos na consulta SQL criada para cada uma delas. No banco desenvolvido, elas foram utilizadas para selecionar, organizar, calcular e filtrar informações armazenadas nas tabelas, tornando a consulta dos dados mais prática e organizada.

STORED PROCEDURES — O QUE FAZ?

As Stored Procedures, também conhecidas como procedimentos armazenados, são conjuntos de comandos SQL que ficam armazenados dentro do próprio banco de dados. Elas são criadas para realizar uma determinada tarefa e podem ser executadas sempre que necessário. Esse recurso permite deixar os comandos mais organizados e facilita a realização de operações que precisam ser repetidas.

Uma Stored Procedure pode ser utilizada para realizar diferentes tipos de operações, como consultar informações, inserir novos registros, alterar dados ou realizar cálculos. Em vez de escrever todos os comandos novamente cada vez que uma tarefa precisa ser realizada, é possível criar uma procedure e executá-la por meio de um comando.

No banco de dados, as Stored Procedures podem trabalhar com as tabelas utilizadas no projeto, como clientes, fornecedores, produtos, pedidos e compras. Dessa forma, elas conseguem utilizar os dados já armazenados para realizar consultas e outras operações.

Uma das principais características das Stored Procedures é a possibilidade de utilizar parâmetros. Os parâmetros permitem que informações sejam enviadas para a procedure no momento em que ela é executada. Isso torna o procedimento mais flexível, pois a mesma estrutura pode ser utilizada para diferentes valores.

Por exemplo, uma procedure pode receber o identificador de um produto e realizar uma consulta para apresentar suas informações. Também pode receber um determinado valor e retornar os produtos que atendem a uma condição definida. Dessa forma, o procedimento pode ser reutilizado em diferentes situações.

A execução de uma Stored Procedure normalmente é realizada utilizando o comando CALL, seguido pelo nome da procedure e, quando necessário, pelos seus parâmetros. Assim, depois que o procedimento é criado, não é necessário escrever novamente todos os comandos SQL que fazem parte dele.

TRIGGERS - O QUE FAZ?

Os Triggers, também chamados de gatilhos, são recursos utilizados em bancos de dados para executar automaticamente uma determinada ação quando ocorre um evento em uma tabela. Diferentemente de uma Stored Procedure, que normalmente precisa ser chamada por um comando, o Trigger é acionado automaticamente de acordo com a condição definida em sua criação.

Os principais eventos que podem ativar um Trigger são o INSERT, utilizado quando um novo registro é inserido; o UPDATE, quando um registro é alterado; e o DELETE, quando um registro é excluído. Também é possível definir se a ação será executada antes ou depois do evento, utilizando BEFORE ou AFTER.

No banco de dados produção, foram criados os Triggers `trg_copia_clientes` e `trg_copia_fornecedores`. Como seus nomes indicam, eles estão relacionados à realização de cópias dos dados de clientes e fornecedores. Esses Triggers trabalham em conjunto com as tabelas principais e as tabelas temporárias existentes no banco, como `clientes_temp` e `fornecedores_temp`.

Esse tipo de recurso é importante porque permite automatizar tarefas que seriam realizadas manualmente. Quando o evento definido no Trigger acontece, o banco executa automaticamente os comandos programados. Isso pode ajudar a manter informações atualizadas, registrar alterações ou realizar cópias de dados.

Por exemplo, considerando um Trigger relacionado à tabela de clientes, quando ocorre o evento definido em sua programação, uma ação pode ser realizada automaticamente na tabela `clientes_temp`. Dessa maneira, não é necessário executar manualmente todos os comandos responsáveis pela cópia das informações.

Os Triggers também podem ser utilizados para manter a organização e a integridade dos dados. Como as ações são executadas automaticamente pelo próprio banco, diminui-se a necessidade de depender de uma ação manual do usuário. Isso é especialmente útil em sistemas que possuem muitas alterações e precisam manter determinadas informações sincronizadas.

É importante destacar a diferença entre Triggers e Stored Procedures. Uma Stored Procedure é executada quando é chamada por meio de um comando, enquanto o Trigger é executado automaticamente quando acontece o evento para o qual foi configurado. Portanto, os dois recursos podem realizar operações no banco, mas possuem formas diferentes de funcionamento.

Durante o desenvolvimento deste trabalho, foi possível compreender melhor o funcionamento dos bancos de dados relacionais e os diferentes recursos utilizados para organizar e manipular informações. As tabelas permitem separar os dados de acordo com suas funções, enquanto os relacionamentos possibilitam trabalhar com informações de diferentes partes do sistema.

Também foi possível compreender a utilização das Views, Stored Procedures e Triggers. As Views facilitam a visualização de dados específicos, as Stored Procedures permitem armazenar e reutilizar conjuntos de comandos SQL e os Triggers possibilitam executar ações automaticamente quando determinados eventos acontecem.

Os conhecimentos adquiridos durante as atividades são importantes para a formação em Programador de Sistemas, pois bancos de dados fazem parte de grande parte dos sistemas desenvolvidos atualmente. A utilização correta desses recursos contribui para criar sistemas mais organizados, eficientes e fáceis de manter.

#	Nome	Tipo	Colação	Atributos	Nulo	Padrão	Comentários	Extra	Ação
☐	1 id	int(11)			Não	Nenhum wrap (padrão: none)		AUTO_INCREMENT	Alterar Eliminar ▼ Mais
☐	2 cod_barras	varchar(100)	latin1_swedish_ci		Sim	NULL			Alterar Eliminar ▼ Mais
☐	3 nome	varchar(255)	latin1_swedish_ci		Não	Nenhum wrap (padrão: none)			Alterar Eliminar ▼ Mais
☐	4 descricao	longtext	latin1_swedish_ci		Sim	NULL			Alterar Eliminar ▼ Mais
☐	5 estoque	float			Não	Nenhum wrap (padrão: none)			Alterar Eliminar ▼ Mais
☐	6 valor	float			Não	Nenhum wrap (padrão: none)			Alterar Eliminar ▼ Mais
☐	7 data_fabricacao	datetime			Sim	NULL			Alterar Eliminar ▼ Mais
☐	8 data_validade	datetime			Sim	NULL			Alterar Eliminar ▼ Mais

☐ Marcar todos
 Com marcados:
 Visualizar
 Alterar
 Eliminar
 Primária
 Único
 Índice
 Texto completo
 Texto completo

Imagem 2: exemplo de tabela - produtos.

#	Nome	Tipo	Colação	Atributos	Nulo	Padrão	Comentários	Extra	Ação
☐	1 id	int(11)			Não	Nenhum wrap (padrão: none)		AUTO_INCREMENT	Alterar Eliminar ▼ Mais
☐	2 nome	varchar(100)	latin1_swedish_ci		Não	Nenhum wrap (padrão: none)			Alterar Eliminar ▼ Mais
☐	3 cpf_cnpj	varchar(18)	latin1_swedish_ci		Não	Nenhum wrap (padrão: none)			Alterar Eliminar ▼ Mais
☐	4 rua	varchar(255)	latin1_swedish_ci		Sim	NULL			Alterar Eliminar ▼ Mais
☐	5 numero	varchar(50)	latin1_swedish_ci		Sim	NULL			Alterar Eliminar ▼ Mais
☐	6 bairro	varchar(100)	latin1_swedish_ci		Sim	NULL			Alterar Eliminar ▼ Mais
☐	7 cep	varchar(10)	latin1_swedish_ci		Sim	NULL			Alterar Eliminar ▼ Mais
☐	8 referencia	varchar(255)	latin1_swedish_ci		Sim	NULL			Alterar Eliminar ▼ Mais
☐	9 telefone	varchar(100)	latin1_swedish_ci		Não	Nenhum wrap (padrão: none)			Alterar Eliminar ▼ Mais

☐ Marcar todos
 Com marcados:
 Visualizar
 Alterar
 Eliminar
 Primária
 Único
 Índice
 Texto completo
 Texto completo

Imagem 3: exemplo de tabela - clientes.